Re module

- (Period): Matches any single character except a newline character.
  On search(a,b)

  e.g:- `print(re.search(r"a.b", "axb")) # Matches 'axb'`

- ^ (Caret): Matches the beginning of the string.
  `E.g:- a=re.search(r"^hello guyz","hello guyz good morning")`
  `If match is found in first  otherwise return none`
  $ (Dollar): Matches the end of the string.

  e.g:- a=re.search(r"world","hello worlds world")

  `If match is found in last  otherwise return none`

- * (Star): Matches zero or more occurrences of the preceding character or group.

  e.g:-a=re.search(r"ab*dfgt","abbbbbdfgt") print(a) o/p:- <re.Match object;

  span=(0, 10), match='abbbbbdfgt'>

- + (Plus): Matches one or more occurrences of the preceding character or group.
  e.g:- a=re.search(r"ab+dfgt","abbbbbdfgt")
  print(a)

- ? (Question Mark): Matches zero or one occurrence of the preceding character or group (makes it optional).

  e.g:- a=re.search(r" roman?t",”romat”)

- {m,n} (Braces): Matches a specified number of occurrences of the preceding character or group, where m is the minimum and n is the maximum.

e.g:- import re print(re.search(r"a{2,4}", "aa")) # Matches 'aa' print(re.search(r"a{2,4}", "aaaa")) # Matches 'aaaa' print(re.search(r"a{2,4}", "a")) # No match

- `[]` (Square Brackets): Defines a character set, matching any single character within the brackets.

e.g:- print(re.search(r"[aeiou]", "hellao")) # Matches 'e' print(re.search(r"[0-9]", "abc123")) # Matches '1'

matches the first occurance in the sequence

- `|` (Pipe): Acts as an OR operator, matching either the expression before or after it.

e.g:-

```
import re
    print(re.search(r"cat|dog", "I have a cat")) # Matches 'cat'
print(re.search(r"cat|dog", "I have a dog")) # Matches 'dog'
```

Python

```
import re
    print(re.search(r"[aeiou]", "hello")) # Matches 'e'
print(re.search(r"[0-9]", "abc123"))  # Matches '1'
```

- `|` (Pipe): Acts as an OR operator, matching either the expression before or after it.

Python

```
import re
    print(re.search(r"cat|dog", "I have a cat")) # Matches 'cat'
print(re.search(r"cat|dog", "I have a dog")) # Matches 'dog'
```

- `()` (Parentheses): Used for grouping subpatterns, capturing matched text, and applying quantifiers to multiple characters.

e.g:- print(re.search(r"(ab)+", "ababab")) # Matches 'ababab' matches the exact sequence in the brackets with the string . or else return none

- **`\` (Backslash)**: Used to escape metacharacters, treating them as literal characters, or to introduce special sequences (e.g., `\d` for digits, `\w` for word characters).

```
e.g:- print(re.search(r"\@", "www.example.@com")) # Matches '@'
print(re.search(r"\%", "www.ex%ample.@com")) # Matches '%'
```

functions in re module

Function	Description
re.search()	Scans an entire string for the *first* location where a pattern matches. It returns a match object upon success and None otherwise.
`re.match()`	Checks for a match only at the *beginning* of the string. This is different from `re.search()`, which scans the entire string.
`re.findall()`	Returns a list of all non-overlapping matches of a pattern in the string. If no matches are found, it returns an empty list.
`re.sub()`	Replaces one or more matches of a pattern with a replacement string and returns the new string.
re.split()	Splits a string by the occurrences of a pattern and returns a list of the resulting substrings.
`re.compile()`	Compiles a regex into a pattern object, which is useful for efficiency if you use the same pattern multiple times in your code

`\d`	Matches any decimal digit.	`[0-9]`
`\D`	Matches any non-digit character.	`[^0-9]`
`\s`	Matches any whitespace character (space, tab, newline, etc.).	`[ \t\n\r\f\v]`
`\S`	Matches any non-whitespace character.	`[^ \t\n\r\f\v]`

`\w`	Matches any alphanumeric character (word character).	`[a-zA-Z0-9_]`
`\W`	Matches any non-word character.	`[^a-zA-Z0-9_]`